Question 1 *(8 points)*

Consider the following MIPS loop executing on a 5-stage pipeline (IF, ID, EX, MEM, WB) with full forwarding and hazard detection. Branches are resolved at the end of the ID stage, so each misprediction squashes one wrongly-fetched instruction (a 1-cycle penalty). Assume $s1 holds a valid base address.

```
Loop:
    lw   $t0, 0($s1)
    lw   $t1, 4($s1)
    add  $t2, $t1, $t0
    sw   $t2, 8($s1)
    addi $s1, $s1, 12
    bne  $s1, $s4, Loop
```

The loop runs 100 iterations. The branch is taken for the first 99 iterations and not taken on the last.

(a) [4 pts] Fill in the pipeline diagram for **one taken iteration** as written, marking stall cycles with — and the squashed fetch slot after the branch. How many cycles does one taken iteration take, counting stalls and the misprediction squash under **predict not-taken**?

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12
`lw $t0`												
`lw $t1`												
`add $t2`												
`sw $t2`												
`addi $s1`												
`bne`												

(b) [**4 pts**] Answer the following:

1. Reorder the instructions in the loop body to eliminate as many stall cycles as possible without changing what the loop computes. Write your reordered sequence and state how many stall cycles remain per iteration.

2. Using your reordered loop and **predict taken**, compute the effective CPI over all 100 iterations. Show your work.

Question 2 *(12 points)*

A 5-stage MIPS pipeline resolves branches in the EX stage, so every misprediction costs a 2-cycle penalty. For a given program: 20% of dynamic instructions are branches, 60% of branches are actually taken, and all non-branch instructions have CPI = 1.

(a) [4 pts] Using the formula CPI = 1 + (branch freq) × (mispredict rate) × (penalty):

1. Compute the effective CPI under (i) predict not-taken and (ii) predict taken.
2. The hardware team proposes moving branch resolution back to the ID stage (1-cycle penalty), but this change would lengthen the clock period by 10%. Using the better predictor from above, is this change worth it? Justify with a speedup calculation.

(b) [4 pts] A single static branch at the bottom of a loop is executed 10 times, producing the outcome sequence below (left to right):

`T T N T T N T T N T`

For each predictor, fill in your predicted outcome for each branch and count the misses.

1. **1-bit history predictor**, starting in the **NT** state.
2. **2-bit history predictor**, starting in the **WT** (weak-taken) state.

Branch #	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Actual outcome	T	T	N	T	T	N	T	T	N	T
1-bit prediction										
2-bit prediction										

(c) [**4 pts**] Answer the following:

1. From part (b), which dynamic predictor performs better on this trace, and what property of the 2-bit predictor explains the difference?

2. This branch follows a repeating `T T N` pattern. Neither predictor can achieve 0% misprediction on it. Briefly explain why.

Question 3 *(8 points)*

Consider a simple system with a single-level page table. The virtual address space is 12-bit, the page size is 256 bytes, and the physical memory has 4 physical frames. Assume a process accesses the following virtual addresses in order: 0x0A3, 0x1F5, 0x0B0. Assume that after a page fault, the OS loads the missing page into a free physical frame and marks that page-table entry valid.

(a) [2 pts] For each virtual address, find its virtual page number. Show your work.

(b) [2 pts] For each virtual address, find its page offset. Show your work.

(c) [4 pts] Suppose the page table initially has no valid entries. Identify which accesses cause a page fault and briefly explain why.

Question 4 *(6 points)*

Consider a byte-addressed system with 8-bit memory addresses. The data cache is direct-mapped, has 32 bytes of data capacity, and uses 8-byte cache blocks. Cache size counts only data bytes, not tag bits or valid bits. The cache is initially empty, so all valid bits are 0. Assume all accesses below are data reads.

The processor accesses the following byte addresses, in order:

`0x10, 0x14, 0x30`.

(a) [**2 pts**] How many cache blocks are in the cache? How many bits are used for the cache index?

(b) [**2 pts**] How many bits are used for the block offset? How many bits are used for the tag?

(c) [**2 pts**] For `0x10`, compute the block address and the block offset. Show your calculations.

MIPS assembly language

Category	Instruction	Example		Meaning	Comments
Arithmetic	add	add	$s1,$s2,$s3	$s1 = $s2 + $s3	Three operands; overflow detected
	subtract	sub	$s1,$s2,$s3	$s1 = $s2 – $s3	Three operands; overflow detected
	add immediate	addi	$s1,$s2,100	$s1 = $s2 + $s3	+ constant; overflow detected
	add unsigned	addu	$s1,$s2,$s3	$s1 = $s2 + $s3	Three operands; overflow undetected
	subtract unsigned	subu	$s1,$s2,$s3	$s1 = $s2 – $s3	Three operands; overflow undetected
	add immediate unsigned	addiu	$s1,$s2,100	$s1 = $s2 + $s3	+ constant; overflow undetected
	move from coprocessor register	mfc0	$s1,$epc	$s1 = $epc	Used to copy Exception PC plus other special registers
	multiply	mult	$s2,$s3	Hi, Lo = $s2 × $s3	64-bit signed product in Hi, Lo
	multiply unsigned	multu	$s2,$s3	Hi, Lo = $s2 × $s3	64-bit unsigned product in Hi, Lo
	divide	div	$s2,$s3	Lo = $s2 / $s3, Hi = $s2 mod $s3	Lo = quotient, Hi = remainder
	divide unsigned	divu	$s2,$s3	Lo = $s2 / $s3, Hi = $s2 mod $s3	Unsigned quotient and remainder
	move from Hi	mfhi	$s1	$s1 = Hi	Used to get copy of Hi
	move from Lo	mflo	$s1	$s1 = Lo	Used to get copy of Lo
Logical	and	and	$s1,$s2,$s3	$s1 = $s2 & $s3	Three reg. operands; logical AND
	or	or	$s1,$s2,$s3	$s1 = $s2 | $s3	Three reg. operands; logical OR
	and immediate	andi	$s1,$s2,100	$s1 = $s2 & 100	Logical AND reg, constant
	or immediate	ori	$s1,$s2,100	$s1 = $s2 | 100	Logical OR reg, constant
	shift left logical	sll	$s1,$s2,10	$s1 = $s2 << 10	Shift left by constant
	shift right logical	srl	$s1,$s2,10	$s1 = $s2 >> 10	Shift right by constant
Data transfer	load word	lw	$s1,100($s2)	$s1 = Memory[$s2+100]	Word from memory to register
	store word	sw	$s1,100($s2)	Memory[$s2 + 100] = $s1	Word from register to memory
	load byte unsigned	lbu	$s1,100($s2)	$s1 = Memory[$s2 + 100]	Byte from memory to register
	store byte	sb	$s1,100($s2)	Memory[$s2 + 100] = $s1	Byte from register to memory
	load upper immediate	lui	$s1,100	$s1 = 100 * 2^{16}	Loads constant in upper 16 bits
Conditional branch	branch on equal	beq	$s1,$s2,25	if ($s1 == $s2) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	bne	$s1,$s2,25	if ($s1 != $s2) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	slt	$s1,$s2,$s3	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than; two's complement
	set less than immediate	slti	$s1,$s2,100	if ($s2 < 100) $s1 = 1; else $s1=0	Compare < constant; two's complement
	set less than unsigned	sltu	$s1,$s2,$s3	if ($s2 < $s3) $s1 = 1; else $s1=0	Compare less than; natural numbers
	set less than immediate unsigned	sltiu	$s1,$s2,100	if ($s2 < 100) $s1 = 1; else $s1 = 0	Compare < constant; natural numbers
Unconditional jump	jump	j	2500	go to 10000	Jump to target address
	jump register	jr	$ra	go to $ra	For switch, procedure return
	jump and link	jal	2500	$ra = PC + 4; go to 10000	For procedure call

Main MIPS assembly language instruction set. The floating-point instructions are shown in Figure 4.47 on page 291. Appendix A gives the full MIPS assembly language instruction set.

MIPS machine language

Name	Format	Example						Comments
		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	
add	R	0	2	3	1	0	32	add $1,$2,$3
sub	R	0	2	3	1	0	34	sub $1,$2,$3
addi	I	8	2	1	100			addi $1,$2,100
addu	R	0	2	3	1	0	33	addu $1,$2,$3
subu	R	0	2	3	1	0	35	subu $1,$2,$3
addiu	I	9	2	1	100			addiu $1,$2,100
mfc0	R	16	0	1	14	0	0	mfc0 $1,$epc
mult	R	0	2	3	0	0	24	mult $2,$3
multu	R	0	2	3	0	0	25	multu $2,$3
div	R	0	2	3	0	0	26	div $2,$3
divu	R	0	2	3	0	0	27	divu $2,$3
mfhi	R	0	0	0	1	0	16	mfhi $1
mflo	R	0	0	0	1	0	18	mflo $1
and	R	0	2	3	1	0	36	and $1,$2,$3
or	R	0	2	3	1	0	37	or $1,$2,$3
andi	I	12	2	1	100			andi $1,$2,100
ori	I	13	2	1	100			ori $1,$2,100
sll	R	0	0	2	1	10	0	sll $1,$2,10
srl	R	0	0	2	1	10	2	srl $1,$2,10
lw	I	35	2	1	100			lw $1,100($2)
sw	I	43	2	1	100			sw $1,100($2)
lui	I	15	0	1	100			lui $1,100
beq	I	4	1	2	25			beq $1,$2,100
bne	I	5	1	2	25			bne $1,$2,100
slt	R	0	2	3	1	0	42	slt $1,$2,$3
slti	I	10	2	1	100			slti $1,$2,100
sltu	R	0	2	3	1	0	43	sltu $1,$2,$3
sltiu	I	11	2	1	100			sltiu $1,$2,100
j	J	2	2500					j 10000
jr	R	0	31	0	0	0	8	jr $31
jal	J	3	2500					jal 10000

MIPS instruction formats

Name	Fields						Comments
Field size	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions 32 bits
R-format	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	op	rs	rt	address/immediate			Transfer, branch, imm. format
J-format	op	target address					Jump instruction format

Main MIPS machine language. Formats and examples are shown, with values in each field: op and funct fields form the opcode (each 6 bits), rs field gives a source register (5 bits), rt is also normally a source register (5 bits), rd is the destination register (5 bits), and shamt supplies the shift amount (5 bits). The field values are all in decimal. Floating-point machine language instructions are shown in Figure 4.47 on page 291. Appendix A gives the full MIPS machine language.